PERFORMANCE IN BIOINFORMATICS SOFTWARE

What constitutes performance and how can it be optimized for Agilent Technologies Enterprise Content Management?

Executive Summary

The engagement of performance testing scenarios using JMeter and the examination of common methods of architecting for performance were implemented to gain a better understanding of the optimal response, scalability, stability and throughput of Agilent Technologies Enterprise Content Management.

Anthem Rukiya J. Wingate

SSW 565 Fall 2019

Table of Contents

Title ..... **Error! Bookmark not defined.**
Executive Summary 1
Introduction 3
Analysis of Tactics 7
Discussion of Applicability 10
Conclusion 16
References 17

Introduction: What is Performance?

Is performance the subjective estimation of end-user perception or the objective measurement of key performance indicators? The collective thought of Ian Molyneax, Len bass, Rick Kazman, and Paul Clements suggests that performance is both the user experience of seamless operation and the system's demonstration of resource availability, throughput, response time and utilization. What is key? Performance is conceived of transactional efficiency. How do the events managed by the system respond to anticipated and unanticipated loads?

For developers, it is of keen interest to ask what is performance for bioinformatics systems as they migrate from financially prohibitive physical distributed network architectures to far more affordable virtualized cloud-based architectures? How is performance negatively affected by anticipated issues such as latency and unanticipated issues such as varying server traffic? More importantly, can bioinformatics software be designed with these issues in mind? How do we architect bioinformatics for performance? Perhaps by first defining performance, this can be achieved.

Expanding the concept of transactional efficiency to quantifiable characteristics is a necessary step to defining performance. The first characteristic to define is the event itself. Events may be defined by how they arrive and what type of resource they consume. An event may arrive at timely intervals, distributed via a statistically identified measure, or randomly. Subsequently events will most commonly be defined as either periodic, stochastic, or sporadic and may each consume such resources as time or threads as the system processes them. These resources must meet specified requirements constraints outlined in the software documentation.

The key outcome to the arrival of an event is optimized resource consumption. As an example, expedited response time may be architected after the system's response to the event is documented and analyzed. What must be documented? First, the event is spoken of as a stimulus and describing how the system responds to that stimulus provides a further dimension of understanding to the architecture based on the quality attribute. In this case, performance as it is shaped by quantifiable characteristics or measurements, turns its focus primarily on latency or processing delay, percentage

of processing deadlines met, throughput or measure of event concurrency, jitter or measure of latency variation, and identification of the number of unprocessed events.

As traceability is an important aspect of defining and designing performance into architecture, six considerations for relating these measurements are collated as a general scenario. The source and destination of the stimulus are identified. The source of the event may either be external or internal while destination is typically the system itself and may be referred to as the artifact. The status of the system at time of arrival and after arrival of the stimulus falls under the heading of environment or response. The observation of environment and response is notated by one of five descriptors: normal, emergency, peak, load or overload. Finally, measurements of latency, throughput, jitter, processing deadlines, and unprocessed events are all collected as response measures.

Andre Bondi notes that there are three key lessons to take into account when determining which metrics are pertinent to the performance testing process.

> "1. One cannot rely on a single number to tell the whole story about the performance of a system.
>
> 2. A description of the performance requirements of a system requires context, such as the offered load (job arrival rate), a description of what is to be done, and the desired response time.
>
> 3. A performance requirement based on an ill-chosen single number is insufficiently specific to tell us how well the system should perform." – (Bondi 2014)

Bondi in turn cites Measuring Computer Performance by David Lilja as the source for a list of the useful properties each performance metric should contain. Notably, the properties which populate Lilja's list bear a familiar resemblance to the list of quality attributes commonly noted in software engineering efforts. These properties are as follows: linearity, reliability, repeatability, consistency, independence and ease of measurement.

For the purposes of examining the performance of Agilent Technologies Enterprise Content Management system, a general scenario will be examined for a handful of relevant events. As this software product evolves into a global cloud-based bioinformatics initiative, resource measures of latency, jitter, and deadlines are of particular concern. This research project intends to implement virtualized environments combined with an edge-computing strategy focused on improving these definitive performance metrics.

According to Molyneaux, the first imperative when gathering performance metrics is that performance testing must be implemented using automated testing tools. Automated tools increase reliability, repeatability and efficiency through implementation of simplified test scripts. Tests can be run with ease as needed and the results may similarly be recorded for ongoing comparison. For these purposes JMeter will be used to test the performance of a virtualized deployment of Agilent Technologies Enterprise Content Management.

JMeter was chosen using Molyneaux's Proof of Concept Checklist. The Proof of Concept checklist has four key areas in which it identifies the usefulness of the chosen performance testing tool. The checklist provides a side by side evaluation of technical suitability between the application and the test scripts. In addition, as pertains to the test scripts, the checklist identifies what input data is necessary for adequate testing of the application. With scripts and input data identified, the checklist assesses the time for script implementation and as may be necessary the time associated with any alterations to the script as additional edge testing opportunities arise. Finally, the Proof of Concept checklist is the best means of demonstrating the full capability of the automated testing tool to adequately test the application.

Prior to carrying out performance testing on Agilent Technologies Enterprise Content Management with JMeter, it was first necessary to identify and list prerequisites, process measures, and deliverables. Prerequisites include an exit criteria, a standard build environment including necessary permissions for the testing tool and application to install successfully and run without interruption, access to the environment limited to those accounts necessary for testing only, input from domain experts as

necessary to facilitate troubleshooting, and sample use cases. Process measures include identification of use cases and the data requirements for all scripts to test for a variety of load scenarios. Finally, the deliverables include what outcomes fall under the equivalence of success or failure, identification of use cases, scripts and data requirements that most effectively test the application.

Once the automated testing tool has been selected, the performance testing process is based upon six generic steps that Molyneaux assures may be applied universally to any project. These steps are to identify the nonfunctional requirements, build the test environment, create scripts for use cases, build the test scenario, execute testing and analyze results and deliver results. These steps are very similar in overall scope to the steps outlined in the Proof of Concept checklist utilized for identifying adequate automated testing tools. Architecting performance as described by Clements, Kazman and Bass coinciding with performance test engineering may become equally granular as is seen in Molyneaux descriptions of the steps involved in designing performance test scenarios with ever increasing specificity. Ideally, performance testing and performance architecting are a co-evolutionary and iterative process in which performance optimization is the outcome of each iterative cycle until the agreed upon requirements for release have been achieved.

Analysis of Tactics

Much like in any system, performance is shaped by the supply and demand of resources. In the case of software, response time is a function of the supply and demand of system resources as periodic, stochastic, and sporadic events arrive at the system individually or concurrently. The response time of an event may be broken into the amount of time it takes for the event to be processed by the system and the amount of time the event may have been delayed from processing by the system. These are referred to as processing time and blocked time. An event may accrue blocked time due to an insufficient availability of resources. If more than one event requires the same resource from the system for processing, those two events are in contention. Blocked time may also accrue if the processing of one event is dependent on the completion of another related event.

Resource availability, contention, and dependency all call for an intentionally performance driven architecture. The design of such an architecture is achieved through the implementation of performance tactics. There are two primary tactics to optimizing response time for performance. The first tactic is to mitigate the system's resource-dependency and the second tactic is to manage the system's resource availability. There are six individual considerations for each of these tactics which in turn may be manipulated with the intent of optimizing response time.

Employing a selection of specific techniques intended to reduce the number of events that must be processed is the manner by which controlling the demand of resources is implemented as a performance architecting tactic. The rate at which data is collected by the system may be manipulated in order to reduce latency. In addition, from placing events into a queue upon arrival to the system emerges the useful technique of conditional prioritization of event execution. What is notably a trade-off for system modifiability, is the latency-reducing technique of linking a single event to a single component so that components which share computational interests in turn share resources. Organizing resources in both proximity and relation to associated components yields a parallel reduction in network communication delay. Placing a limit on the amount of time available for system response and improving algorithm effectiveness in direct proportion to the level of criticality, are two final means of latency reduction.

Likewise, there are several techniques that may be implemented as a performance architecting tactic which are intended to control the availability of resources. Resource availability may most readily be managed by providing more resources via implementation of non-functional system requirements. In addition, resources may be managed via parallel utilization of resources, redundancy in copies of computations and data, limiting event arrivals, and organizing resource utilization via schedule.

When event arrival outpaces resource availability, queuing occurs. How the system handles queuing can be a significant means by which performance is tested and ultimately, by which performance is designed into the architecture. The laws that determine the relationship between performance of the system and the queuing behavior of events are built entirely on definitions of time or the reciprocal measure of rate. The time between each event is the interarrival time. Likewise, the time utilized to resource the event is the service time. It is expected that with differing loads on the system, these times will vary. As a result, queues may form when the interarrival time is less than the service time or when variances in interarrival rate and service rate result in an event arriving while an event is being processed. The system must be architected with an expectation of variability. If queues are expected, buffers can be designed to hold events until they can be scheduled

.

Andre Bondi asserts that the performance of a system can be broken down to its simplest and most reliable measures using the basic definitions for timing, queuing and the laws of performance. Once a metric for the mean service time has been reported for a system, the utilization can be calculated by multiplying this measure by the throughput. Throughput is reported as transactions per unit of time. The formula for this calculation is defined as the Utilization Law and may be used to determine the percentage of utilization, more commonly known as the saturation level of the system. It is generally accepted that a system should not exceed eighty percent saturation as levels above eighty percent will negatively impact system stability.

Given the average throughput and average response time, the product of these two measures is the average queue length. The formula for this calculation is defined as Little's law. Average queue length is also a function of traffic intensity. Traffic intensity is the product of the process rate for an event and the mean service time for all events. When the traffic intensity is greater than or equal to one, the

system is definitively saturated, or the number of events arriving at the system is consistently greater than the number of events leaving.

The average queue length is equal to a ratio of the traffic intensity divided by one minus the traffic intensity. Ideally, traffic intensity is mathematically in equilibrium. This is expressed as less than one and greater than or equal to zero. The percent saturation of the system, if the events are queued "first come first serve" or "last come first serve" are specifications that can be used to build a testing scenario in conjunction with the load weight for the test. The load weight may be light, moderate or heavy. Average response time, average queue length, average arrival rate and average service time all inform queuing behavior. In turn, queuing behavior can be specified in testing to lend further insight into how the system may be designed for improved resource utilization.

Discussion of Applicability

In relation to architecting for the purpose of controlling resource demand and managing resource availability, Agilent Technologies Enterprise Content Management begins with key non-functional requirements for minimum RAM and storage specifications in order to optimize responsiveness. Scalability and throughput are addressed by placing limits on concurrent licensed users. System stability is addressed by separating key system activities across multiple APIs. For example, data transported from networked clients into SQL database storage on a centralized server are handled by the file transfer service. The system activities are tracked and prioritized by the application service. Likewise, security measures for the system are handled by OpenLab Shared Services and access to storage via network protocol is handled by Java API.

Testing the effectiveness of the design choice using the JMeter GUI is very straightforward. The performance testing scenario is transferred to the JMeter platform via creation of the thread group, samplers, logical controllers and listeners. This is accomplished using a user interface and intuitive dialogue boxes. There also exist templates for gaining a better understanding of the Web API you are testing. For example, the recording template can be implemented in any situation where the source code of the software being tested is not readily available while clarifying the exact manner that test data must be input. The recording template identifies the input variables so that they may be placed directly into the test script. The testing tool was then run on the Agilent Technologies Enterprise Content Management web server posting the necessary login information gained from the recording template directly to the html form.

The following testing scenarios were implemented using the JMeter tool on Agilent Technologies Enterprise Content Management:

System Description:

> Enterprise Content Management (by Agilent Technologies) version 3.6 deployed as an all in one server in a Windows 2016 VM utilizing SQL 2017 on the backend. Enterprise Content Management using a web client for access via Internet Explorer, latest version compatible. Tested using JMeter 5.1.1.

Test Plan:

- Thread Groups Utilized (3 Scenarios)
  - 1 Thread/ 1 Second
  - 6 Threads / 60 Seconds
  - 30 Threads/60 Seconds
- Once Only Logical Controller
  - HTTP Request – "ECM Login"
  - Server/IP
    - http://win-43g480f7bpe/ECM/
  - Post
    - User name
    - Password
    - Account
    - Domain
- Listener
  - Graph Results – "ECM Login Results"
  - View Results Tree

Workload Model:

- Scenario
  - User browses ECM Content
- Action
  - Login
  - Browse
  - Conduct search for all pdf files

Log out

Input Data

User name

Password

Account

Domain

Output Data

Content Page

Think Time

5 to 10 seconds

Design Considerations

Number of Users: 1, 6, 30 (based on number of accounts registerd in ECM system)

Iteration per minute: 1, 6, 30 (during key usage times, all users will be logged into ECM system)

End to End Response Time (Iteration Response Time): 60 seconds

Number of Transactions: 6

Think Time: 5 to 10 seconds

Hits (Requests) per second: 0.1

Throughput: moderate

The results of this test plan for a sample population of thirty yields an average service time of 167 milliseconds. The throughput was recorded as 0.52 transactions per second with the system performing under a moderate load. For this test scenario, server utilization is calculated to be 0.087. The average response time was a recorded 2.82 seconds. Thus the average queue length shall then be calculated to be 1.4664 events. If the arrival rate for the scenario is roughly 0.024 events per second based on the calculated average queue length, then the calculated traffic intensity is 0.0041. For a

moderate load, this measure of traffic intensity reveals that the system is not at all saturated. A reasonable expectation based on these calculations would be that even under heavy load, system saturation should reliably not exceed the acceptable measure of eighty percent given that a 100 percent saturated system correlates with a traffic intensity equal to one. To prove that these scenarios are a relevant predictor for how the system would operate under heavy load, test scenarios should be run that mimic systems with hundreds of concurrent global users. The outcome of test scenarios of the system under heavy load is meant to substantially increase the measures for throughput and average queue length for the purpose of comparing the effect on the service time, the response time and the interarrival time.

The outcomes of the test scenarios may be used in conjunction with the performance design checklist to determine key architectural decisions. First the system's responsibilities in association with criticality and load are identified and coordinated in a model. The coordination of these responsibilities must meet response time specifications, handle any anticipated concurrency, respond to all events and retain associated properties of pertinent communication mechanisms such as whether the events are synchronous or asynchronous. This model is expanded with data in order to determine which techniques for data handling and abstraction and how mapping the components in specific architectures should be implemented. Next resources are managed based on criticality, binding times and utilized technologies are reviewed for their impact. For additional insight into the testing results, please see the supplementary documentation attached to the end of this paper.

Bioinformatics applications are inherently high-fidelity data intensive systems. As a result, architecting performance into these systems means managing resources rather than reducing the demand on resources. Bass, Kazman and Clements identify the most effective means of managing resources is by increasing their availability. The key means by which to accomplish this is through scheduling and prioritization of queued events or implementing better system resourcing by replicating data and resources associated with those events. A final means by which to manage resources is by establishing non-functional requirements that set specifications for RAM that better provision the system for most anticipated user scenarios.

Henry Liu speaks of hardware architectures that can lend to performance optimization. In the case of Agilent Technologies Enterprise Content Management, the use of N-Tier Architecture is a first step in this direction. This provides for a client tier and a web tier for access to the network distributed system, a database tier for organization of the data that is collected, and an application tier that supports N components. The N components may include an application server, and multiple network accessed storage locations. This is the foundation on which Agilent Technologies Enterprise Content Management was built at its inception. With performance optimization in mind, this entire hardware architecture is placed on virtual machines that are maintained and operate with greater efficiency and availability of resources.

The N-Tier architecture design has proven to be an optimal solution for customers operating on a global scale. However, for customers with smaller interests this has proven to have a largely negative impact in that it becomes financial prohibitive even in a virtualized environment. Each gigabyte of space in the cloud or on a hosted virtual server costs the customer. Spinning up multiple servers, each meeting the minimum non-functional requirements for hardware, can evolve into an unnecessary expense. This may be true in particular if the customer spins up a two terabyte hard drive per specification, but only uses three to five hundred gigs for operating system and temporary cache. That is an additional 1.5 terabytes of storage space they are paying for even though they are not putting that space to any use.

This is one of two readily recognizable limits to the capability of implementing tactics for performance optimization of Agilent Technologies Enterprise Content Management. The other limit is to customers installing Agilent Technologies Enterprise Content Management in cloud-based topologies like Microsoft's Azure. The absence of full domain architecture including a domain controller and DNS is a fatal deterrent to Agilent Technologies Enterprise Content Management system operation. Agilent Technologies Enterprise Content Management is configured on the basis of Windows authentication using ASP.Net and IIS. Thus the absence of full domain topology is not a currently supported deployment of the software. This means that customers unwilling or financially unable to build their full network topology in AWS or Azure, may have to rely solely on the long-supported technologies provided by a physical network paired with virtual machines.

In spite of these limitations, there is room to take steps towards optimal performance for Agilent Technologies Enterprise Content Management system. Supporting all in one Agilent Technologies Enterprise Content Management servers with domain-independent configurations ready to deploy as a super-light topology for the cloud, is the sole recommendation for the next steps in engineered performance optimization. Providing a domain-independent solution that is not reliant on ASP.Net and IIS for Windows Authentication will allow customers to take the secondary step of moving their entire bioinformatics system to a cloud-based topology. Though this may be the sole pertinent recommendation, it is currently the biggest evolutionary step for the software presently. An all in one cloud-based server may take a step away from the optimization provided by N-Tier architecture, it takes a simultaneous step towards a virtual machine that can be provisioned with all necessary resource availability in mind while offering the customer substantial financial savings.

Conclusion

"Every system has real-time performance constraints." This statement is offered for discussion in Bass, Clements and Kazman's Software Architecture in Practice. In order to architect a system while building in performance optimization, it is necessary to answer the question, what is performance? Further, the real-time performance constraints of the system must be identified in order to architect an effective means by which to mitigate those constraints. This paper has endeavored to define performance with increasing precision while holding true a central tenant that performance is an identified measure of transactional efficiency. Looking with greater depth and understanding into this definition yields how performance is measured and how optimal performance is achieved as a step-wise practice.

Engineering performance is itself a process that first begins with querying the system via performance tests, and architecting solutions using pre-established techniques for mitigating the demand for while increasing the availability of system resources. Agilent Technologies Enterprise Content Management system was not initially designed, nor has it evolved, in a vacuum. As a product, it is currently coming of age in a diverse peer group of data intensive software. Agilent Technologies Enterprise Content Management and its competitors in the market must hold up what may be considered the key metric of indisputably high levels of performance and that is low latency. As software systems move toward diminished footprint and cloud topologies, Agilent Technologies Enterprise Content Management must refactor and thus adapt its configuration requirements to meet the expectations of what is quickly becoming industry standard. The real-time performance constraints with these new expectations are that Agilent Technologies Enterprise Content Management must expand operational requirements to include single-server architectures and architectures that function independent of a domain controller or domain name system.

In the text Foundations of Software and System Performance Engineering, Andre Bondi reminds readers that there exist quite a few pitfalls to architecting performance into any system. It matters little whether the intended improvements are at the behest of stakeholders or to edge out competitors in the market. Bondi points out that increasing resource availability may have unintended consequences when it comes to scheduling and resource prioritization. Equally likely, proposed changes may fall under the umbrella of diminishing returns in terms of new technologies that promise ever speedier processing power. In reality, engineers may discover that their efforts to architect

performance may be unknowingly skewed by the metrics themselves. What happens when time-stamping information which documents response time for resource utilization in real-time systems is itself measured incorrectly? When the measures that shape the performance model are faulty, any architecting decisions dictated so too are faulty.

Bondi reminds that performance engineering should be undertaken with healthy concern that every attempt to make a system more efficient may reveal or create an unknown issue that diminishes the overall optimization effort. Expanding the definition of performance from a measure of transactional efficiency encourages exploration beyond surface understanding of performance optimization through architecture. This means confronting the realization that system refactoring techniques such as resource queueing are not the only impetus by which performance is impacted. Any vehicle the business organization utilizes to make the product more successful in the market may also impact performance. Thus, Bondi offers a useful lesson in finding a comprehensive definition of performance. That is this, designing architecture on the basis of performance is an effort which must be made with "vigilance and skepticism". Performance engineering isn't about reshaping an entire system, so much as it is about improving the overall efficiency of a system while making the smallest possible impact upon it.

References

15 Best Performance Testing Tools (Load Testing Tool) in 2019
https://www.softwaretestinghelp.com/performance-testing-tools-load-testing-tools/

The Art of Application Performance Testing 2nd Edition, by Molyneaux, Ian published by O'Reilly Media, Inc 2014. Chapters 1-5

Foundations of Software System Performance Engineering: Process, Performance Modeling, Requirements, Testing Scalability Practice by Andre B. Bondi, published by Addison-Wesley Professional 2014.

Software Architecture in Practice 3rd Edition, by Bass, Clements and Kazman published by Addison Wesley 2012. Chapter 8: Performance pp. 131-146

Software Performance and Scalability: A Quantitative Approach by Henry H. Liu, published by Wiley-Blackwell 2009. Chapter 1: 1.3 Testing Software Performance and Scalability